

8. CI/CD Pipeline Documentation

8.1 Introduction

Continuous Integration (CI) and Continuous Deployment (CD) are modern software development practices that automate code integration, testing, and deployment activities.

The purpose of implementing CI/CD in this project is to automate the execution of Selenium test cases whenever source code changes are pushed to the GitHub repository. This ensures immediate feedback on application quality and helps identify defects early in the development lifecycle.

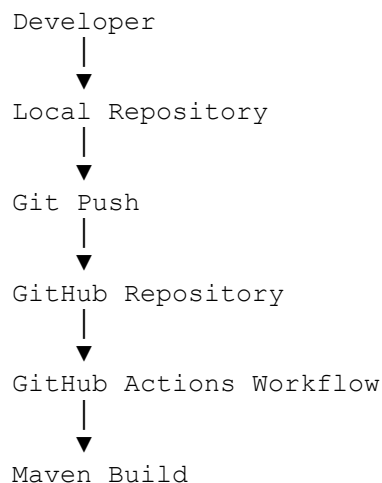
GitHub Actions was selected as the CI/CD platform because it integrates seamlessly with GitHub repositories and supports automated workflow execution.

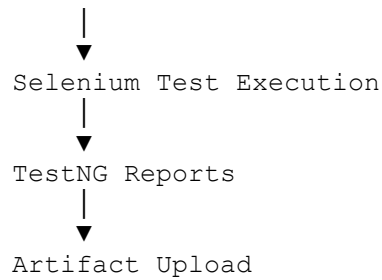
8.2 CI/CD Objectives

The primary objectives of CI/CD implementation are:

- Automate test execution.
 - Reduce manual intervention.
 - Improve software quality.
 - Detect defects early.
 - Enable continuous testing.
 - Generate automated reports.
 - Support rapid software delivery.
-

8.3 CI/CD Architecture





8.4 Tools Used in CI/CD Pipeline

Tool	Purpose
Git	Version Control
GitHub	Source Code Repository
GitHub Actions	CI/CD Automation
Maven	Build Execution
Selenium	Test Automation
TestNG	Test Execution
Java	Programming Language

8.5 Workflow Trigger Conditions

The workflow executes automatically when:

Push Event

```
on:
  push:
    branches:
      - main
```

Whenever code is pushed to the main branch.

Pull Request Event

```
pull_request:
  branches:
    - main
```

Whenever a pull request is created or updated.

8.6 Workflow File Structure

Location:

```
.github/workflows/selenium-ci.yml
```

Purpose:

Defines all CI/CD execution steps.

8.7 Workflow Configuration

Step 1 – Checkout Repository

```
- name: Checkout Repository
  uses: actions/checkout@v4
```

Downloads repository source code.

Step 2 – Setup Java

```
- name: Setup Java
  uses: actions/setup-java@v4
```

Installs Java 21.

Step 3 – Build Project

```
- name: Build Project
  run: mvn clean test
```

Executes:

- Maven Build
 - Selenium Tests
 - TestNG Execution
-

Step 4 – Upload Reports

```
- name: Upload Test Reports
```

Uploads execution reports as GitHub Artifacts.

8.8 Complete Workflow File

```
name: Parabank Selenium CI

on:
  push:
    branches:
      - main
      - master

  pull_request:
    branches:
      - main
      - master

jobs:
  test:

    runs-on: ubuntu-latest

    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Setup Java 21
        uses: actions/setup-java@v4
        with:
          distribution: 'temurin'
          java-version: '21'

      - name: Build Project
        run: mvn clean test

      - name: Upload Test Reports
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: TestNG-Reports
          path: test-output/
```

8.9 CI/CD Execution Flow

1. Developer pushes code.
2. GitHub detects changes.
3. GitHub Actions workflow starts.
4. Source code is downloaded.
5. Java environment is configured.
6. Maven dependencies are installed.
7. Selenium test scripts execute.

8. TestNG reports are generated.
 9. Reports uploaded as artifacts.
 10. Workflow status displayed.
-

8.10 Benefits of CI/CD Implementation

Faster Feedback

Developers receive immediate test results.

Early Defect Detection

Defects identified before deployment.

Automated Regression Testing

Regression suite executes automatically.

Improved Software Quality

Consistent testing process.

Reduced Manual Effort

No manual execution required.

8.11 Challenges Faced During Implementation

- GitHub authentication setup.
 - Git repository configuration.
 - GitHub Actions workflow creation.
 - Browser execution in headless mode.
 - Dependency compatibility management.
 - Report artifact configuration.
-

8.12 CI/CD Validation Results

The pipeline was validated successfully.

Results:

- ✓ Repository checkout successful
 - ✓ Java setup successful
 - ✓ Maven build successful
 - ✓ Selenium execution successful
 - ✓ TestNG reports generated
 - ✓ Artifacts uploaded successfully
-

8.13 Conclusion

The CI/CD pipeline was successfully implemented using GitHub Actions for the Parabank Automation Framework. The pipeline automates build execution, Selenium testing, report generation, and artifact management. This implementation demonstrates how continuous integration practices can improve software quality, reduce manual effort, and enable efficient automated testing within modern software development environments.